3-5 · Pandas와 DuckDB를 이용한 하이브리드 가공 · 실습 1.5H

정제 파이프라인 구현

기준선을 재고, 함수로 정제하고, 두 번 돌린다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① DuckDB 로 orders.csv 의 품질 기준선(before)을 잰다

② clean_orders.py — 추출은 DuckDB, 정제는 Pandas 로 쓴다

③ 두 번 돌려 equals() 가 True 인지(멱등성) 확인한다

④ validate() 를 만들고 일부러 깨뜨려 실제로 잡는지 본다

결과물 orders_clean.parquet 와 clean_orders.py 가 다음 1H 리포트의 재료입니다

실습 흐름

여섯 단계 — ~/data-eng/week3 한 폴더에서

3-1 의 orders.csv 가 없으면 make_orders.py 를 다시 실행합니다 — 시드가 고정돼 언제 만들어도 같은 원본

1. 기준선	2. clean()	3. 두 번 돌리기	4. 순서 실험	5·6. 검증·완성
DuckDB 로 before 측정	clean_orders.py 1015 → 1000 행	equals() True 세 번	중앙값 A·B 비교	validate() 종료 코드

1단계

원본의 품질 기준선 — baseline.py

rows	count(*) → 1015
null_price	count(*) - count(price) → 30
dup_orders	count(*) - count(DISTINCT order_id) → 15
category_values	count(DISTINCT category) → 5
오염 확인	GROUP BY category — 식품 396 과 ' 식품 ' 20 이 따로

이 수치가 리포트의 before — 화면에선 구분이 안 되는 공백을 DB 는 다른 값으로 본다 (일관성 문제의 전형)

2단계

clean() 다섯 줄 — 한 줄씩 뜻

줄	뜻
str.strip()	category 공백 오염 제거
pd.to_datetime()	DuckDB 면 이미 날짜 — read_csv 대비
drop_duplicates()	order_id 기준, 첫 행 유지
fillna(median())	price 결측을 중앙값으로
price * quantity	amount 파생 컬럼

load_raw() 는 duckdb.sql("SELECT * FROM 'orders.csv'").df() — 대용량이면 여기서 필터·열 선택으로 줄인다

df.copy() 로 시작 — out = df 면 받은 원본까지 바뀐다(pandas 3 도). 3단계 「두 번 돌려도 같은가」가 여기에 기댄다

2·3단계

돌리고, 두 번 돌리고 — 출력을 읽는다

1. **python3 clean_orders.py**
   「1015 행 → 1000 행 정제 완료」 — 중복 15건 제거, orders_clean.parquet 생성

2. **python3 check_rerun.py**
   행 수 같은가 · 내용 같은가(equals) · 파일로도 같은가 — 세 줄 모두 True

3. **하나라도 False 면**
   정제 규칙에 무작위성이나 순서 의존이 섞인 것 — t1·t2.parquet 는 rm 으로 정리

확인할 글자

True

세 번

4단계

순서를 바꾸면 중앙값이 달라진다 — 실제로

1. **순서 A — 중복 제거 후 중앙값**
   drop_duplicates(subset='order_id') 뒤 median() → 100500.0

2. **순서 B — 중복 있는 채로 중앙값**
   raw.copy() 그대로 median() → 101000.0

3. **같은가: False**
   중복 15건이 두 번 세어져 가운데 값이 500원 밀린다

order_test.py

실제로
다르다

> 결측 30건을 어느 값으로 채우는지가 순서에 따라 바뀐다 — 「중복 제거 → 결측 보정」 순서를 고정하고 리포트에 적는다

5단계 — 일부러 실패시키기

검사기를 만들었으면 일부러 깨뜨린다

validate_test.py — 지금 데이터엔 없지만, 원본이 바뀌었을 때 잡아 줍니다

1. validate()	2. 정제본 검사	3. 나쁜 값 주입	4. 다시 검사
음수 가격·수량 미래날짜·미지 분류	네 항목 모두 0 = 유효성 통과	price -1000 category '알수없음'	price_le_0 1 unknown_cat 1

6단계

최종본으로 통째로 바꾼다 — 실패면 종료 코드 1

바꾸기	clean_orders.py 를 최종본으로 통째로 (검증 포함)
validate() 5항목	price_le_0 · quantity_le_0 · unknown_cat · dup_order_id · null_price
실패하면	print('검증 실패:', issues) → raise SystemExit(1)
통과하면	orders_clean.parquet 저장 · 「정제 완료 (검증 통과)」
확인	python3 clean_orders.py; echo "종료 코드: $?" → 0

2단계 파일에 이어 붙이지 않는다 · VALID_CATEGORIES = 식품·가전·의류·도서 · 8주차 Airflow 가 이 숫자를 본다

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. baseline.py — 1015 · 30 · 15 · 5, ' 식품 ' 20 이 따로
2. 「1015 행 → 1000 행 정제 완료」· orders_clean.parquet 생성
3. check_rerun.py — 세 줄 모두 True
4. order_test.py — 100500.0 vs 101000.0 · 같은가: False
5. validate_test.py — 첫 줄 모두 0, 나쁜 값엔 price_le_0 1 · unknown_cat 1
6. 최종본 실행 뒤 「검증 통과」와 종료 코드 0

3-5 · 실습 1.5H 마무리

다음은 데이터 품질 리포트 작성 (1H)

정제 전후를 같은 잣대로 재서 리포트를 만들고, DuckDB 로 교차 검증한 뒤 3주차 산출물로 제출합니다

- ✓ **quality_report.py — before·after 프로파일**
- ✓ **개선율 네 줄**
- ✓ **DuckDB 로 교차 검증**
- ✓ **리포트 문서 작성 · 산출물 제출**

가져갈 것

orders_clean.parquet 와 스크립트